

Code Output

A. Intermediate Calculations

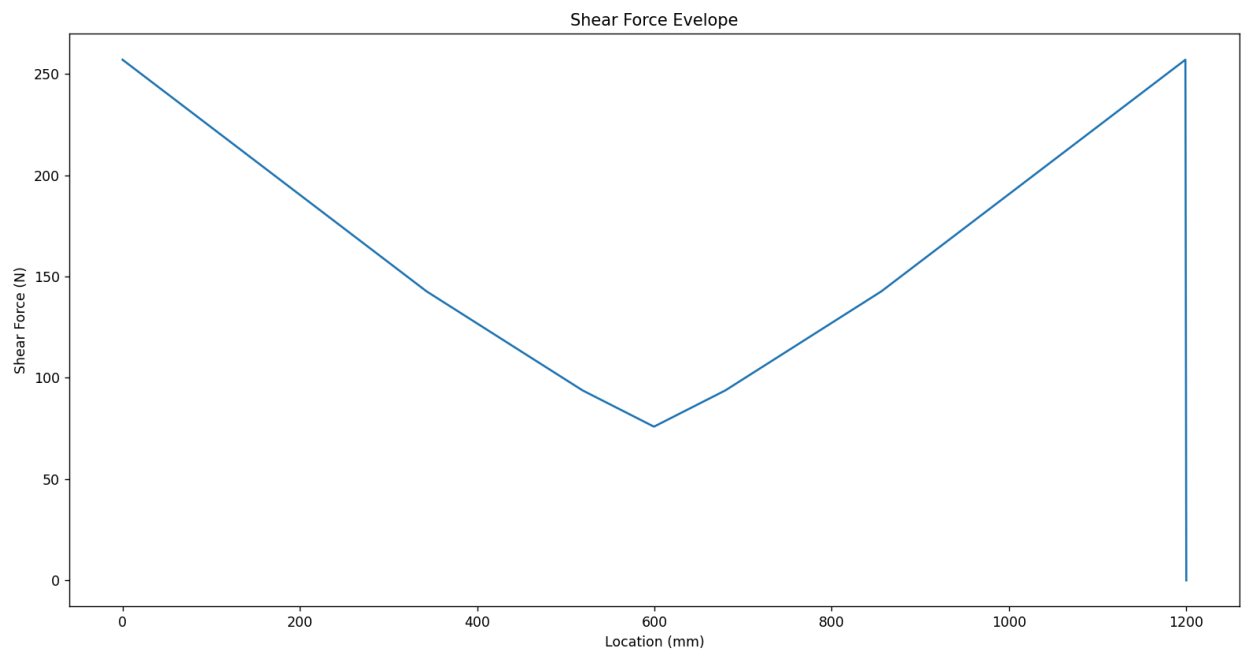
Load case 1 maximum bending moment: 69260

Load case 1 maximum shear force: 257

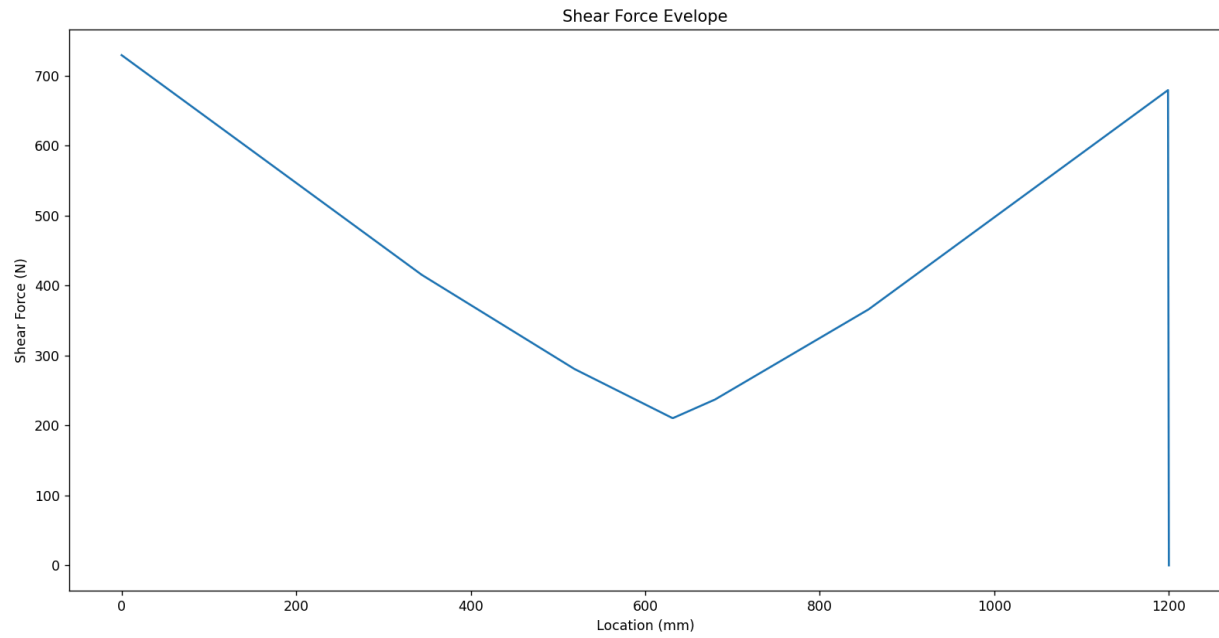
B. SFE

Self-weights of bridges are neglected. All graphs start at the centre of one support and end at the centre of the other support. This is because the wheels start after the centre of the support, as seen in the assignment handout, thus no shear force or moment will be present in the bridge before and after the supports. In load case 2, the locomotive is assumed to be the rightmost car.

Load Case 1:

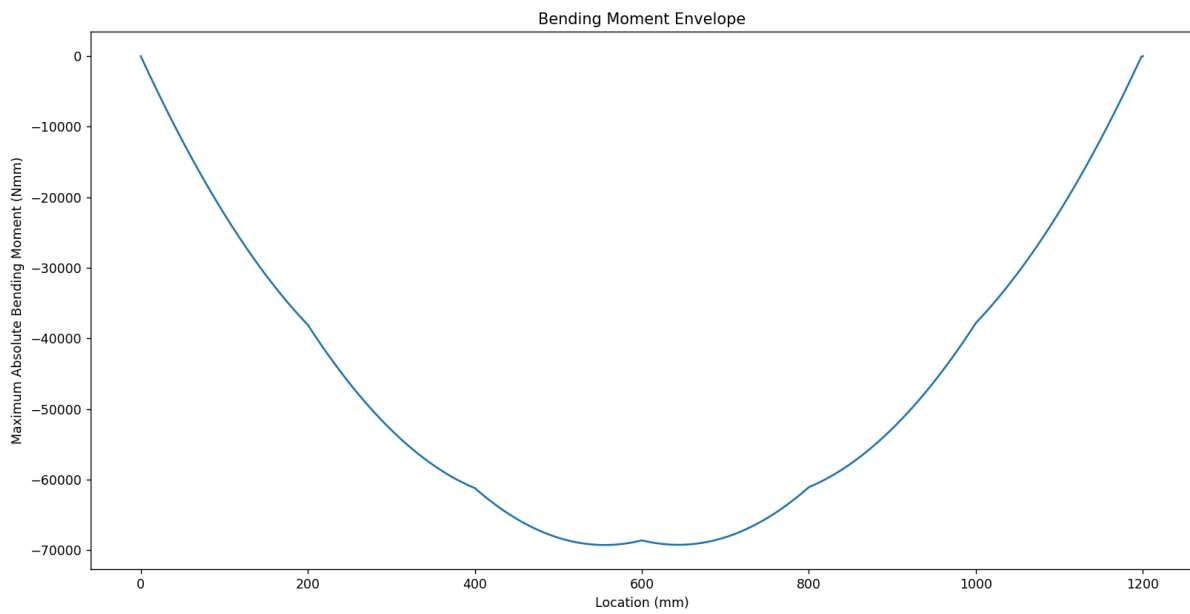


Load Case 2 Failure Load:

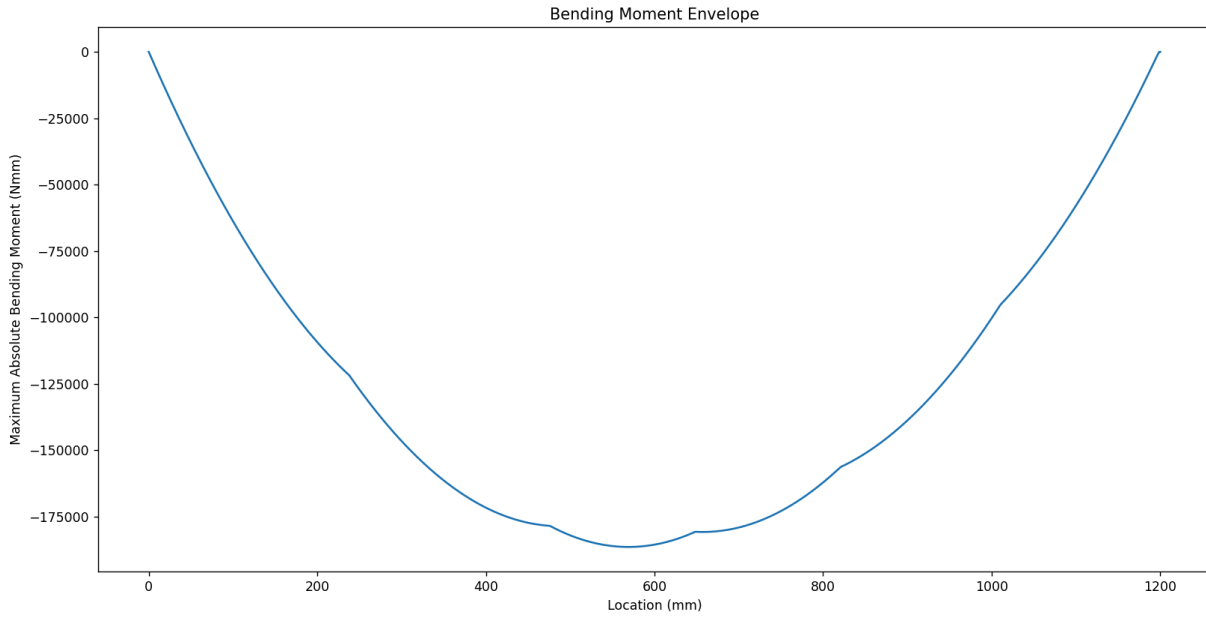


C. BME

Load Case 1:



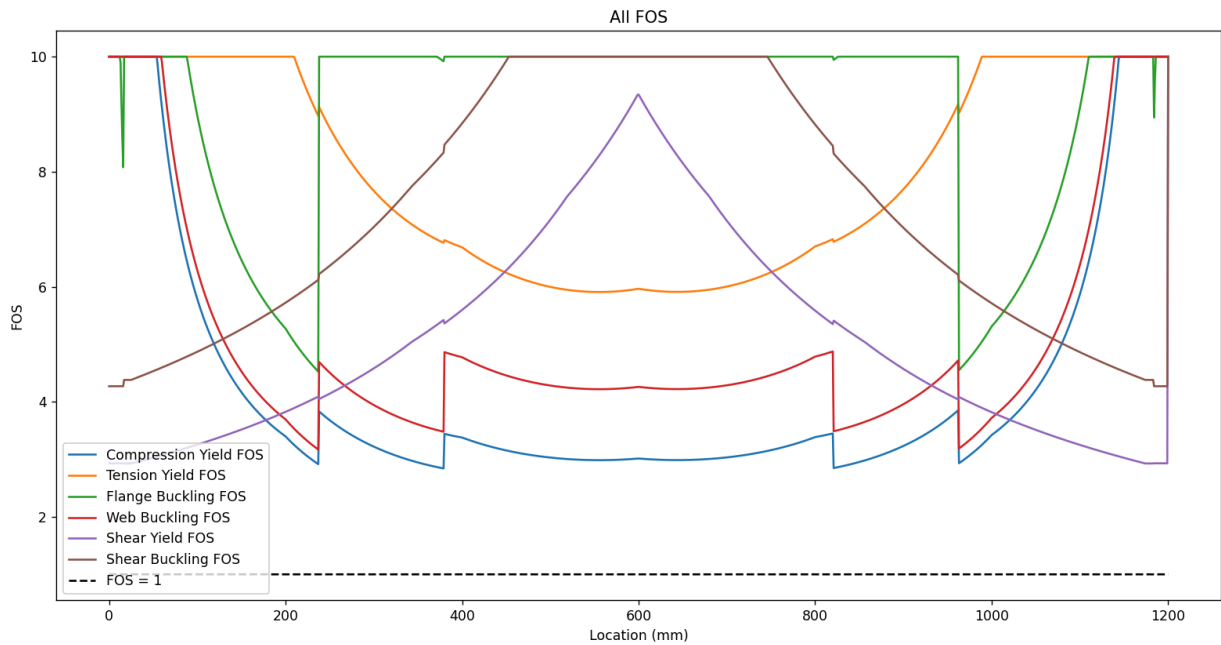
Load Case 2 Failure Load:



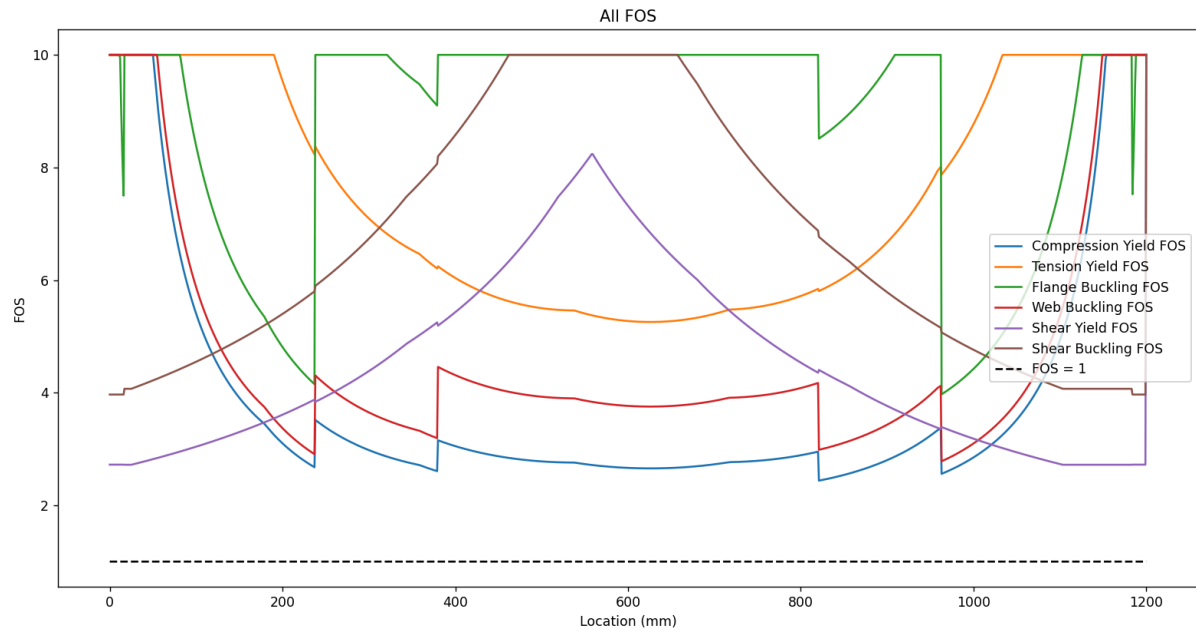
D. FOS

Note that FOS is limited to not display values greater than 10 for visualization purposes.

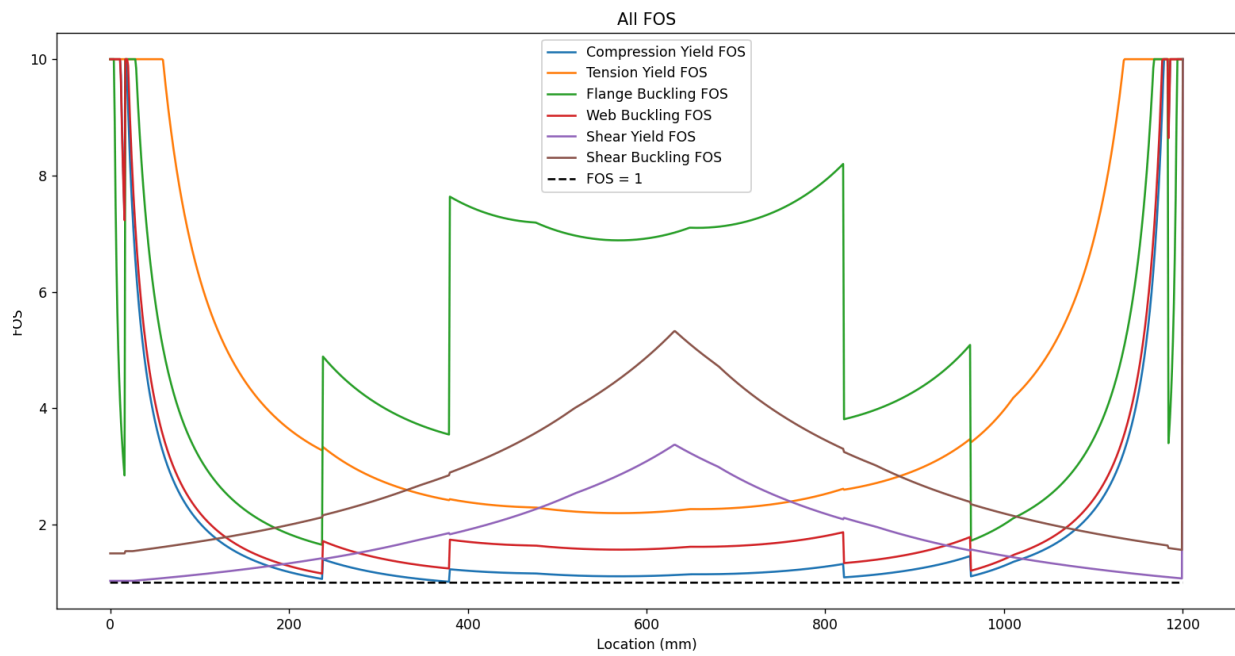
Load Case 1:



Load Case 2 Base Case:



Load Case 2 Failure Load:



E. Entire Script

Language: Python

Package used: matplotlib

```
coefficients = []

# note that for bridge lengths, only the portion between
# the supports is considered (0 to 1200 mm)
def load_case(n):
    global coefficients
    if n == 1:
        coefficients = [[1/6, 0], [1/6, 176], [1/6, 340], [1/6, 516],
                        [1/6, 680], [1/6, 856]]
        # in this case, P is the total load
    elif n == 2:
        # load case 2:
        # assume last car is locomotive
        # middle car is freight car of load P
        # rightmost car is 1.1P, leftmost car is 1.38P
        # load distribution:
        # 0.55 176 0.55 164 0.5 176 0.5 164 0.69 176 0.69
        # supports at 0 and 1200
        # let x be the distance travelled by the front wheels of the
train
        coefficients = [[0.69, 0], [0.69, 176], [0.5, 340], [0.5, 516],
                        [0.55, 680], [0.55, 856]]

    elif n == 3:
        # load case 2 base case
        # use P = 1
        # 135 135 182
        coefficients = [[67.5, 0], [67.5, 176], [67.5, 340], [67.5,
516],
                        [91, 680], [91, 856]]

# calculates reactions at A and B for given x (location of backwheel)
and P
def supports_reactions(x, P):
    MA, MB = 0, 0
```

```

    for c in coefficients:
        x_i = x + c[1]
        if 0 <= x_i <= 1200:
            MA += c[0] * P * x_i
            MB += c[0] * P * (1200 - x_i)
    return MB/1200, MA/1200

# returns array for shear force for every integer length given x and P
# at a given train displacement
def shear_force(x, P):
    A, B = supports_reactions(x, P)
    V = []
    for i in range(1201):
        V_x = A
        for c in coefficients:
            x_i = x + c[1]
            if 0 <= x_i <= i:
                V_x -= c[0] * P
        V.append(V_x)
    V[1200] += B
    return V

# returns M, an array of bending moments for each mm
# at a given train displacement
def bending_moment(x, P):
    V = shear_force(x, P)
    M = [0 for i in range(1201)]
    for i in range(1, 1201):
        M[i] = -V[i] + M[i-1]
    return M

# returns bending moment envelope, an array of the maximum bending
moment of each mm
# under all possible displacements of the train
def get_BME(P):
    BME = [0 for i in range(1201)]
    for x in range(-856, 1201):
        M = bending_moment(x, P)
        for i in range(1201):

```

```

        if BME[i] > M[i]:
            BME[i] = M[i]

    return BME

# returns shear force envelope, an array of the maximum shear force of
# each mm
# under all possible displacements of the train; uses absolute values
def get_SFE(P):
    SFE = [0 for i in range(1201)]
    for x in range(-856, 1201):
        V = shear_force(x, P)
        for i in range(1201):
            if SFE[i] < abs(V[i]):
                SFE[i] = abs(V[i])
    return SFE

# draws a diagram for visualization
def get_diagram(M, title="Bending Moment Envelope", ylabel="Maximum
Absolute Bending Moment (Nmm)":
    import matplotlib.pyplot as plt
    plt.plot([i for i in range(1201)], M)
    plt.title(title)
    plt.xlabel("Location (mm)")
    plt.ylabel(ylabel)
    plt.show()

# gets failure load from a given max M (positive value); only works for
# load case 2
# returns total load in N
def get_failure_load(max_M):
    # binary search for P
    P_low, P_high = 0, 2000
    while P_high - P_low > 0.1: # precision to 0.1
        P_mid = (P_low + P_high) / 2
        max_M_with_P_mid = -min(get_BME(P_mid))
        if max_M_with_P_mid < max_M: # note that BME is negative
            P_low = P_mid
        else:
            P_high = P_mid

```

```

    return ((P_low + P_high) / 2) * 3.48 # calculate total load

# calculates the FOSs for a cross section to fail in bending
def get_flexural_FOS(t, b1, b2, h, n, M):
    # t: thickness of flanges and web
    # b1: top flange width
    # b2: bottom flange width
    # h: web height
    # n: number of layers of additional material below top flange
    # all length units in mm
    # M: actual bending moment in Nmm
    # avoid division by zero
    if M == 0:
        return [float('inf'), float('inf'), float('inf'), float('inf')]
    # calculate cross section properties
    A1, A2, A3, A4 = b1*t, 2*h*t, n*t*(b2 - 2*t), b2*t
    y1, y2, y3, y4 = h+3*t/2, h/2+t, h+t-n*t/2, t/2
    ybar = (A1*y1 + A2*y2 + A3*y3 + A4*y4) / (A1 + A2 + A3 + A4)
    I = ((b1+b2+n**3*(b2-2*t))*t**3 + 2*t*h**3) / 12 \
    + A1*(y1 - ybar)**2 + A2*(y2 - ybar)**2 + A3*(y3 - ybar)**2 + A4*(y4
- ybar)**2
    # calculate stresses from material ult and thin plate buckling;
    # units in MPa
    sigma_buckle_flange = (4*3.1416**2*4000)/(12*(1-0.2**2)) *
((t+t*n)/(b2-2*t))**2
    sigma_buckle_web = (6*3.1416**2*4000)/(12*(1-0.2**2)) *
(t/(h-n*t))**2
    # calculate allowable moments in Nmm
    M_comp = 6 * I / (h+2*t - ybar)
    M_buckle_flange = sigma_buckle_flange * I / (h+2*t - ybar)
    M_buckle_web = sigma_buckle_web * I / (h - n*t - ybar)
    M_tens = 30 * I / ybar
    return [M_comp/M, M_tens/M, M_buckle_flange/M, M_buckle_web/M]
# FOS for [compression yield, tension yield, flange buckle, web buckle]

# calculates FOS for a cross section to fail to shear
def get_shear_FOS(t, b1, b2, h, n, V):
    # check for division by 0
    if V == 0:

```

```

        return float('inf')

    # cross sectional properties
    A1, A2, A3, A4 = b1*t, 2*h*t, n*t*(b2 - 2*t), b2*t
    y1, y2, y3, y4 = h+3*t/2, h/2+t, h+t-n*t/2, t/2
    ybar = (A1*y1 + A2*y2 + A3*y3 + A4*y4) / (A1 + A2 + A3 + A4)
    I = ((b1+b2+n**3*(b2-2*t))*t**3 + 2*t*h**3) / 12 \
    + A1*(y1 - ybar)**2 + A2*(y2 - ybar)**2 + A3*(y3 - ybar)**2 + A4*(y4
- ybar)**2

    # calculate Q
    yQ = ((ybar-t)*(ybar+t)*t + A4*y4) / (2*(ybar-t)*t + A4)
    Qcent = (2*(ybar-t)*t + A4) * (ybar - yQ) # Qg doesn't matter
because glue tabs

    # calculate shear buckling
    a = 150
    tau = (5*3.1416**2*4000)/(12*(1-0.2**2)) * ((t/(h-n*t))**2 +
(t/(a))**2)

    return [(8*I*t)/(Qcent*V), ((2*tau*I*t)/Qcent)/V]
# FOS for [material shear, buckling shear]

# calculates the geometry of the final design
def design_geometry():
    # calculate length of each layer required
    t, b1, b2, h = 1.27, 120, 100, 74
    BME = get_BME(288) # 1kN
    SFE = get_SFE(288)
    n = 0
    while 1:
        x1, x2 = -1, -1
        for i in range(1201):
            if min(get_flexural_FOS(t, b1, b2, h, n, abs(BME[i]))) < 1:
                x1 = i
                break
        for i in range(1200, -1, -1):
            if min(get_flexural_FOS(t, b1, b2, h, n, abs(BME[i]))) < 1:
                x2 = i
                break
        if x1 == -1 and x2 == -1:
            break

```

```

        print(n, x1, x2)
        # number of layers, min FOS, first failure point, last failure
point
        n += 1
        print("FOS for shear:", get_shear_FOS(t, b1, b2, h, 1, abs(SFE[0])))

# calculates and plots all FOS at every point
# uses failure load
def plot_all_FOS(t, b1, b2, h, P):
    BME, SFE = get_BME(P), get_SFE(P)
    # set up arrays to store FOS
    comp, tens, flange, web, shear, buckle_shear = [], [], [], [], [],
[]

    minFOS = float('inf')
    minFOS_location = -1
    # loop through every point
    for x in range(1201):
        n = 0
        if 380 <= x <= 820:
            n = 3
        elif 238 <= x <= 962:
            n = 2
        elif 17 <= x <= 1183:
            n = 1

        # retrieve FOS values from functions
        [a, b, c, d] = get_flexural_FOS(t, b1, b2, h, n, -BME[x])
        [e, f] = get_shear_FOS(t, b1, b2, h, n, min(SFE[x], SFE[25])) #
diaphragms at support

        # limit max FOS to 10 for better visualization
        comp.append(min(a, 10))
        tens.append(min(b, 10))
        flange.append(min(c, 10))
        web.append(min(d, 10))
        shear.append(min(e, 10))
        buckle_shear.append(min(f, 10))

        # check if any FOS is below 1
        if min(a, b, c, d, e, f) < 1:
            print("Failure at x =", x)
        if min(a, b, c, d, e, f) < minFOS:

```

```

        minFOS = min(a, b, c, d, e, f)
        minFOS_location = x
    print("Minimum FOS across bridge:", minFOS, "at x =",
minFOS_location)

    # plot all diagrams
    import matplotlib.pyplot as plt
    plt.plot([i for i in range(1201)], comp, label="Compression Yield
FOS")
    plt.plot([i for i in range(1201)], tens, label="Tension Yield FOS")
    plt.plot([i for i in range(1201)], flange, label="Flange Buckling
FOS")
    plt.plot([i for i in range(1201)], web, label="Web Buckling FOS")
    plt.plot([i for i in range(1201)], shear, label="Shear Yield FOS")
    plt.plot([i for i in range(1201)], buckle_shear, label="Shear
Buckling FOS")
    plt.plot([i for i in range(1201)], [1 for i in range(1201)], 'k--',
label="FOS = 1")
    plt.legend()
    plt.title("All FOS")
    plt.xlabel("Location (mm)")
    plt.ylabel("FOS")
    plt.show()

# running the code
if __name__ == "__main__":

    # cross sectional dimensions for final design
    t, b1, b2, h = 1.27, 120, 100, 77.5

    # calculate for load case 1:
    load_case(1)

    # BME and SFE
    BME, SFE = get_BME(400), get_SFE(400)
    get_diagram(BME)
    get_diagram(SFE, title="Shear Force Evelope", ylabel="Shear Force
(N) ")

```

```

# intermediate calculations for load case 1
print("Load case 1 maximum bending moment:", -min(BME))
print("Load case 1 maximum shear force:", SFE[0])

plot_all_FOS(t, b1, b2, h, 400)

# calculate for load case 2:
load_case(2)

# calculate failure load
P = 315

# BME and SFE
BME, SFE = get_BME(P), get_SFE(P)
get_diagram(BME)
get_diagram(SFE, title="Shear Force Evelope", ylabel="Shear Force
(N) ")

# calculate FOS across every point
plot_all_FOS(t, b1, b2, h, P)

# calculating for final design:
# design_geometry()

# load case 2 base case
load_case(3)
BME, SFE = get_BME(1), get_SFE(1)
print("Load case 2 base case maximum bending moment:", -BME[821])
print("Load case 2 base case maximum shear force:", SFE[821])
plot_all_FOS(t, b1, b2, h, 1)

```